lab6

```m
function lab6()
% ================================================================
% File: lab6.m
% Course: Queueing Systems - Lab 6
% Topic: Time-Series Forecasting
%
% Basic part only - no validation set:
%   1. Try 5 Linear Regression models
%   2. Try 5 Filter models
%   3. Select the best model in each category using median CRPS
%   4. Plot only:
%      - best Linear Regression model
%      - best Filter model
%      - comparison between the two selected models
%
% Required file:
%   dataset.mat must be in the same directory as this .m file.
% ================================================================

close all;
clc;

% Octave needs the statistics package for norminv, normcdf, normpdf.
if exist('pkg', 'builtin') || exist('pkg', 'file')
  try
    pkg load statistics;
  catch
    warning('Could not load Octave statistics package. Make sure norminv, normcdf and normpdf are available.');
  end
end

% ================================================================
% 1. Load dataset
% ================================================================

dataset_file = 'dataset.mat';

data = load(dataset_file);

if ~isfield(data, 'values')
  error('dataset.mat does not contain a variable named values.');
end

y = double(data.values(:));

n = length(y);
nob = 1200;                 % first 50 days = 1200 hours
alpha = 0.05;               % 95% prediction interval
z = norminv(1 - alpha / 2);  % approximately 1.96
```

```
50
51  if n <= nob
52    error('The dataset must contain more than 1200 observations.');
53  end
54
55  nforecast = n - nob;
56  t_all = (1:n)';
57  forecast_idx = (nob + 1:n)';
58
59  fprintf('\n============================================================\n');
60  fprintf('Lab 6 - Basic Forecasting with Model Selection\n');
61  fprintf('============================================================\n');
62  fprintf('Loaded dataset: %s\n', dataset_file);
63  fprintf('Number of observations: %d\n', n);
64  fprintf('Training observations:  %d\n', nob);
65  fprintf('Forecast observations:  %d\n', nforecast);
66  fprintf('Forecast horizon:       %d to %d\n', nob + 1, n);
67  fprintf('Confidence level:       95%%\n');
68
69  % ============================================================
70  % 2. Evaluate 5 Linear Regression models
71  % ============================================================
72
73  reg_results(1) = evaluate_regression_model(y, nob, alpha, ...
74    'LR1: constant + daily 24', ...
75    0, [24]);
76
77  reg_results(2) = evaluate_regression_model(y, nob, alpha, ...
78    'LR2: linear trend + daily 24', ...
79    1, [24]);
80
81  reg_results(3) = evaluate_regression_model(y, nob, alpha, ...
82    'LR3: linear trend + daily 24 + weekly 168', ...
83    1, [24, 168]);
84
85  reg_results(4) = evaluate_regression_model(y, nob, alpha, ...
86    'LR4: quadratic trend + daily 24 + weekly 168', ...
87    2, [24, 168]);
88
89  reg_results(5) = evaluate_regression_model(y, nob, alpha, ...
90    'LR5: linear trend + daily 24 + half-daily 12 + weekly 168', ...
91    1, [24, 12, 168]);
92
93  best_reg_idx = find_best_by_crps(reg_results);
94  best_reg = reg_results(best_reg_idx);
95
96  % ============================================================
97  % 3. Evaluate 5 Filter models
98  % ============================================================
99
100 filter_results(1) = evaluate_filter_model(y, nob, alpha, ...
101   'F1: Delta_24', ...
102   make_diff_filter([24]));
103
```

```
filter_results(2) = evaluate_filter_model(y, nob, alpha, ...
  'F2: Delta_1 Delta_24', ...
  make_diff_filter([1, 24]));

filter_results(3) = evaluate_filter_model(y, nob, alpha, ...
  'F3: Delta_168', ...
  make_diff_filter([168]));

filter_results(4) = evaluate_filter_model(y, nob, alpha, ...
  'F4: Delta_1 Delta_168', ...
  make_diff_filter([1, 168]));

filter_results(5) = evaluate_filter_model(y, nob, alpha, ...
  'F5: Delta_24 Delta_168', ...
  make_diff_filter([24, 168]));

best_filter_idx = find_best_by_crps(filter_results);
best_filter = filter_results(best_filter_idx);

% ============================================================
% 4. Print comparison tables
% ============================================================

fprintf('\n============================================================\n');
fprintf('Linear Regression Candidate Models\n');
fprintf('============================================================\n');
fprintf('%-60s %8s %12s %12s %12s %12s\n', ...
  'Model', 'Params', 'Train MSE', 'MAE', 'RMSE', 'Med CRPS');

for i = 1:length(reg_results)
  fprintf('%-60s %8d %12.3f %12.3f %12.3f %12.3f\n', ...
    reg_results(i).name, ...
    reg_results(i).num_params, ...
    reg_results(i).train_mse, ...
    reg_results(i).mae, ...
    reg_results(i).rmse, ...
    reg_results(i).median_crps);
end

fprintf('\nBest Linear Regression model by median CRPS:\n');
fprintf('  %s\n', best_reg.name);

fprintf('\n============================================================\n');
fprintf('Filter Candidate Models\n');
fprintf('============================================================\n');
fprintf('%-35s %8s %12s %12s %12s %12s\n', ...
  'Model', 'Order', 'Res Std', 'MAE', 'RMSE', 'Med CRPS');

for i = 1:length(filter_results)
  fprintf('%-35s %8d %12.3f %12.3f %12.3f %12.3f\n', ...
    filter_results(i).name, ...
    filter_results(i).order, ...
    filter_results(i).residual_std, ...
    filter_results(i).mae, ...
```

```
    filter_results(i).rmse, ...
    filter_results(i).median_crps);
end

fprintf('\nBest Filter model by median CRPS:\n');
fprintf('  %s\n', best_filter.name);

fprintf('\n============================================================\n');
fprintf('Final Selected Models Comparison\n');
fprintf('============================================================\n');
fprintf('%-35s %12s %12s %12s %12s\n', ...
  'Selected model', 'MAE', 'RMSE', 'Coverage', 'Med CRPS');

fprintf('%-35s %12.3f %12.3f %11.2f%% %12.3f\n', ...
  best_reg.name, ...
  best_reg.mae, ...
  best_reg.rmse, ...
  100 * best_reg.coverage, ...
  best_reg.median_crps);

fprintf('%-35s %12.3f %12.3f %11.2f%% %12.3f\n', ...
  best_filter.name, ...
  best_filter.mae, ...
  best_filter.rmse, ...
  100 * best_filter.coverage, ...
  best_filter.median_crps);

if best_reg.median_crps < best_filter.median_crps
  fprintf('\nOverall, the selected Linear Regression model has better median
CRPS.\n');
elseif best_filter.median_crps < best_reg.median_crps
  fprintf('\nOverall, the selected Filter model has better median CRPS.\n');
else
  fprintf('\nThe selected models have equal median CRPS.\n');
end

write_results_csv('lab6_model_comparison_results.csv', reg_results, filter_results);

% ================================================================
% 5. Plots only for the selected best models
% ================================================================

save_figures = true;
npast = min(168, nob - 1);
past_idx = (nob - npast:nob)';

% Figure 1: Dataset and train-test split
figure(1);
plot(t_all(1:nob), y(1:nob), 'k');
hold on;
plot(t_all(forecast_idx), y(forecast_idx), 'o-b');
yl = ylim;
line([nob nob], yl, 'linestyle', '--', 'color', 'k');
grid on;
```

```
legend('Training data', 'Forecast/test data', 'Train-test split', ...
       'location', 'northwest');
title('Dataset and train-test split');
xlabel('Hour');
ylabel('GPU resource requests');
hold off;

% Figure 2: Best regression fit on training data
figure(2);
plot(t_all(1:nob), y(1:nob), 'k');
hold on;
plot(t_all(1:nob), best_reg.yhat(1:nob), 'r');
grid on;
legend('Training data', 'Best linear regression fit', ...
       'location', 'northwest');
title(['Best linear regression fit: ', best_reg.name]);
xlabel('Hour');
ylabel('GPU resource requests');
hold off;

% Figure 3: Best regression forecast
figure(3);
plot(t_all(past_idx), y(past_idx), 'k');
hold on;
plot(t_all(forecast_idx), y(forecast_idx), 'o-b');
plot(t_all(forecast_idx), best_reg.yhat(forecast_idx), 'r');
plot(t_all(forecast_idx), best_reg.upper, 'r-.');
plot(t_all(forecast_idx), best_reg.lower, 'r-.');
grid on;
legend('Past observed data', 'True future values', ...
       'Best regression forecast', 'Upper 95% PI', 'Lower 95% PI', ...
       'location', 'northwest');
title(['Best linear regression forecast: ', best_reg.name]);
xlabel('Hour');
ylabel('GPU resource requests');
hold off;

% Figure 4: Best filter residual diagnostics
figure(4);
res_x = (best_filter.order + 1:nob)';
plot(res_x, best_filter.filtered_residuals_centered, 'b');
hold on;
line([res_x(1) res_x(end)], [0 0], 'linestyle', '--', 'color', 'k');
grid on;
legend('Centered filtered residuals', 'Zero mean reference', ...
       'location', 'northwest');
title(['Best filter residual diagnostics: ', best_filter.name]);
xlabel('Hour');
ylabel('Filtered residual');
hold off;

% Figure 5: Best filter forecast
figure(5);
plot(t_all(past_idx), y(past_idx), 'k');
```

```
hold on;
plot(t_all(forecast_idx), y(forecast_idx), 'o-b');
plot(t_all(forecast_idx), best_filter.yhat(forecast_idx), 'r');
plot(t_all(forecast_idx), best_filter.upper, 'r-.');
plot(t_all(forecast_idx), best_filter.lower, 'r-.');
grid on;
legend('Past observed data', 'True future values', ...
       'Best filter forecast', 'Upper 95% PI', 'Lower 95% PI', ...
       'location', 'northwest');
title(['Best filter forecast: ', best_filter.name]);
xlabel('Hour');
ylabel('GPU resource requests');
hold off;

% Figure 6: Point forecast comparison
figure(6);
plot(t_all(forecast_idx), y(forecast_idx), 'o-k');
hold on;
plot(t_all(forecast_idx), best_reg.yhat(forecast_idx), 'r');
plot(t_all(forecast_idx), best_filter.yhat(forecast_idx), 'b');
grid on;
legend('True future values', 'Selected regression forecast', 'Selected filter
forecast', ...
       'location', 'northwest');
title('Point forecast comparison of selected models');
xlabel('Hour');
ylabel('GPU resource requests');
hold off;

% Figure 7: Absolute error comparison
figure(7);
plot(t_all(forecast_idx), abs(best_reg.errors), 'r');
hold on;
plot(t_all(forecast_idx), abs(best_filter.errors), 'b');
grid on;
legend('Selected regression absolute error', 'Selected filter absolute error', ...
       'location', 'northwest');
title('Absolute forecast error comparison of selected models');
xlabel('Hour');
ylabel('Absolute error');
hold off;

% Figure 8: Common forecast comparison with 95% prediction intervals
%
% This figure directly satisfies the requirement of showing, in one common
% graph, the true future values, the forecasts, and the 95% prediction
% intervals for the two selected methods.
figure(8);
plot(t_all(past_idx), y(past_idx), 'k');
hold on;

plot(t_all(forecast_idx), y(forecast_idx), 'o-k');

plot(t_all(forecast_idx), best_reg.yhat(forecast_idx), 'r');
```

```
plot(t_all(forecast_idx), best_reg.upper, 'r-.');
plot(t_all(forecast_idx), best_reg.lower, 'r-.');

plot(t_all(forecast_idx), best_filter.yhat(forecast_idx), 'b');
plot(t_all(forecast_idx), best_filter.upper, 'b-.');
plot(t_all(forecast_idx), best_filter.lower, 'b-.');

grid on;
legend('Past observed data', 'True future values', ...
       'Selected regression forecast', 'Regression upper 95% PI', 'Regression lower 95% PI', ...
       'Selected filter forecast', 'Filter upper 95% PI', 'Filter lower 95% PI', ...
       'location', 'northwest');
title('Selected models forecast comparison with 95% prediction intervals');
xlabel('Hour');
ylabel('GPU resource requests');
hold off;

if save_figures
  print(1, 'lab6_fig1_dataset_train_test.png', '-dpng');
  print(2, 'lab6_fig2_best_regression_fit.png', '-dpng');
  print(3, 'lab6_fig3_best_regression_forecast.png', '-dpng');
  print(4, 'lab6_fig4_best_filter_residuals.png', '-dpng');
  print(5, 'lab6_fig5_best_filter_forecast.png', '-dpng');
  print(6, 'lab6_fig6_selected_point_forecast_comparison.png', '-dpng');
  print(7, 'lab6_fig7_selected_absolute_error_comparison.png', '-dpng');
  print(8, 'lab6_fig8_selected_forecast_intervals_comparison.png', '-dpng');

  % Short aliases, useful for the LaTeX report if you are using lab6_1.png, ..., lab6_8.png.
  print(1, 'lab6_1.png', '-dpng');
  print(2, 'lab6_2.png', '-dpng');
  print(3, 'lab6_3.png', '-dpng');
  print(4, 'lab6_4.png', '-dpng');
  print(5, 'lab6_5.png', '-dpng');
  print(6, 'lab6_6.png', '-dpng');
  print(7, 'lab6_7.png', '-dpng');
  print(8, 'lab6_8.png', '-dpng');

  fprintf('\nSaved figures as PNG files in the current directory.\n');
end

fprintf('\nSaved model comparison metrics to lab6_model_comparison_results.csv.\n');
fprintf('Execution completed successfully.\n');

end

% ============================================================
% Subfunctions
% ============================================================

function result = evaluate_regression_model(y, nob, alpha, model_name, degree, periods)

```

```
    n = length(y);
    z = norminv(1 - alpha / 2);
    forecast_idx = (nob + 1:n)';

    X_all = build_regression_matrix(n, degree, periods);
    X_train = X_all(1:nob, :);

    p = size(X_train, 2);

    b = X_train \ y(1:nob);
    yhat = X_all * b;

    residuals = y(1:nob) - X_train * b;
    train_mse = sum(residuals .^ 2) / nob;

    denom = max(nob - p, 1);
    sigma2 = sum(residuals .^ 2) / denom;
    sigma = sqrt(sigma2);

    G = pinv(X_train' * X_train);

    nforecast = n - nob;
    cint = zeros(nforecast, 1);

    for i = 1:nforecast
      x0 = X_all(nob + i, :);
      g = x0 * G * x0';
      cint(i) = z * sigma * sqrt(1 + g);
    end

    lower = yhat(forecast_idx) - cint;
    upper = yhat(forecast_idx) + cint;

    [mae, rmse, coverage, median_crps, crps, errors] = ...
      compute_forecast_metrics(y(forecast_idx), yhat(forecast_idx), lower, upper, z);

    result.name = model_name;
    result.type = 'regression';
    result.degree = degree;
    result.periods = periods;
    result.num_params = p;
    result.coefficients = b;
    result.yhat = yhat;
    result.lower = lower;
    result.upper = upper;
    result.cint = cint;
    result.residuals = residuals;
    result.train_mse = train_mse;
    result.mae = mae;
    result.rmse = rmse;
    result.coverage = coverage;
    result.median_crps = median_crps;
    result.crps = crps;
    result.errors = errors;
```

```
end

function X = build_regression_matrix(n, degree, periods)

  t = (1:n)';
  tau = t / 24;  % time measured in days for numerical scaling

  p = degree + 1 + 2 * length(periods);
  X = zeros(n, p);

  col = 1;

  for k = 0:degree
    X(:, col) = tau .^ k;
    col = col + 1;
  end

  for j = 1:length(periods)
    T = periods(j);

    X(:, col) = cos(2 * pi * t / T);
    col = col + 1;

    X(:, col) = sin(2 * pi * t / T);
    col = col + 1;
  end

end

function result = evaluate_filter_model(y, nob, alpha, model_name, filter_coeffs)

  n = length(y);
  z = norminv(1 - alpha / 2);
  forecast_idx = (nob + 1:n)';
  nforecast = n - nob;

  a = filter_coeffs(:)';
  q = length(a) - 1;

  if a(1) == 0
    error('The first filter coefficient must be nonzero.');
  end

  if q >= nob
    error('Filter order is too high for the available training set.');
  end

  raw_filtered = apply_filter_to_training(y(1:nob), a);

  residual_mean = mean(raw_filtered);
  centered_residuals = raw_filtered - residual_mean;
  residual_std = std(centered_residuals);

```

```
  yhat = y;
  cint = zeros(nforecast, 1);

  imp = [1; zeros(nforecast - 1, 1)];
  inv_h = filter(1, a, imp);

  for k = nob + 1:n
    horizon = k - nob;

    prediction = residual_mean;

    for j = 1:q
      prediction = prediction - a(j + 1) * yhat(k - j);
    end

    prediction = prediction / a(1);
    yhat(k) = prediction;

    sigma_h = residual_std * sqrt(sum(inv_h(1:horizon) .^ 2));
    cint(horizon) = z * sigma_h;
  end

  lower = yhat(forecast_idx) - cint;
  upper = yhat(forecast_idx) + cint;

  [mae, rmse, coverage, median_crps, crps, errors] = ...
    compute_forecast_metrics(y(forecast_idx), yhat(forecast_idx), lower, upper, z);

  result.name = model_name;
  result.type = 'filter';
  result.coefficients = a;
  result.order = q;
  result.yhat = yhat;
  result.lower = lower;
  result.upper = upper;
  result.cint = cint;
  result.filtered_residuals_raw = raw_filtered;
  result.filtered_residuals_centered = centered_residuals;
  result.residual_mean = residual_mean;
  result.residual_std = residual_std;
  result.mae = mae;
  result.rmse = rmse;
  result.coverage = coverage;
  result.median_crps = median_crps;
  result.crps = crps;
  result.errors = errors;

end

function res = apply_filter_to_training(y_train, a)

  q = length(a) - 1;
  n = length(y_train);
  m = n - q;
```

```
531
532     res = zeros(m, 1);
533
534     for i = 1:m
535       t = q + i;
536       value = 0;
537
538       for j = 0:q
539         value = value + a(j + 1) * y_train(t - j);
540       end
541
542       res(i) = value;
543     end
544
545   end
546
547   function a = make_diff_filter(lags)
548
549     a = 1;
550
551     for i = 1:length(lags)
552       L = lags(i);
553
554       if L < 1
555         error('All differencing lags must be positive integers.');
556       end
557
558       b = [1, zeros(1, L - 1), -1];
559       a = conv(a, b);
560     end
561
562   end
563
564   function [mae, rmse, coverage, median_crps, crps, errors] = ...
565     compute_forecast_metrics(y_true, mu, lower, upper, z)
566
567     y_true = y_true(:);
568     mu = mu(:);
569     lower = lower(:);
570     upper = upper(:);
571
572     errors = y_true - mu;
573
574     mae = mean(abs(errors));
575     rmse = sqrt(mean(errors .^ 2));
576     coverage = mean((y_true >= lower) & (y_true <= upper));
577
578     sigma = (upper - mu) / z;
579     sigma(sigma <= 0) = eps;
580
581     crps = zeros(length(y_true), 1);
582
583     for i = 1:length(y_true)
584       a = (y_true(i) - mu(i)) / sigma(i);
```

```
    crps(i) = sigma(i) * ...
      (a * (2 * normcdf(a) - 1) + 2 * normpdf(a) - 1 / sqrt(pi));
  end

  median_crps = median(crps);

end

function idx = find_best_by_crps(results)

  values = zeros(length(results), 1);

  for i = 1:length(results)
    values(i) = results(i).median_crps;
  end

  [~, idx] = min(values);

end

function write_results_csv(filename, reg_results, filter_results)

  fid = fopen(filename, 'w');

  if fid < 0
    warning('Could not write results CSV file.');
    return;
  end

  fprintf(fid,
'category,model,train_mse_or_residual_std,mae,rmse,coverage,median_crps\n');

  for i = 1:length(reg_results)
    fprintf(fid, 'regression,"%s",%.10f,%.10f,%.10f,%.10f,%.10f\n', ...
      reg_results(i).name, ...
      reg_results(i).train_mse, ...
      reg_results(i).mae, ...
      reg_results(i).rmse, ...
      reg_results(i).coverage, ...
      reg_results(i).median_crps);
  end

  for i = 1:length(filter_results)
    fprintf(fid, 'filter,"%s",%.10f,%.10f,%.10f,%.10f,%.10f\n', ...
      filter_results(i).name, ...
      filter_results(i).residual_std, ...
      filter_results(i).mae, ...
      filter_results(i).rmse, ...
      filter_results(i).coverage, ...
      filter_results(i).median_crps);
  end

  fclose(fid);

```

```
638 end
```